DATABASE MANAGEMENT SYSTEM LAB

ASSIGNMENT NO.6

Case study: Airline Management System

Mini Project: Centralized Club Management System

Name: Nishtha Pramod Parve

PRN: 124B1F003

Case Study: Airline Management System

Aim: Write all types of Database Triggers for suitable application.

```
mysql> USE airline_management;

Database changed

mysql> SHOW TABLES;

+------------------------------+

| Tables_in_airline_management |

+------------------------------+

| airline                      |

| airline_copy                 |

| booking                      |

| flight                       |

| passenger                    |

+------------------------------+

5 rows in set (0.04 sec)


mysql> select * from airline;

+------------+--------------+---------+--------------
+

| airline_id | airline_name | country |
founded_year |

+------------+--------------+---------+--------------
+

|          1 | Air India    | India   |         NULL |

|          2 | Indigo       | India   |         NULL |

|          3 | Spicejet     | India   |         NULL |

|          4 | Airindia     | India   |         NULL |

+------------+--------------+---------+--------------
+
```

```
4 rows in set (0.02 sec)


mysql> select * from booking;

+------------+-----------+--------------+------------
-+---------+--------------+

| booking_id | flight_id | passenger_id |
booking_date | seat_no | travel_class |

+------------+-----------+--------------+------------
-+---------+--------------+

|       1001 |       101 |          501 | 2024-02-
01   | 12A     | Economy      |

|       1002 |       105 |          789 | 2023-09-
08   | 14B     | Economy      |

|       1003 |       123 |          890 | 2025-07-
07   | 13A     | Economy      |

+------------+-----------+--------------+------------
-+---------+--------------+

3 rows in set (0.01 sec)


mysql> select * from flight;

+-----------+------------+---------+------------+---
------------+--------------+------------+-----------
--+-----------+

| flight_id | airline_id | source  |
destination | departure_time |
arrival_time | flight_type | total_seats |
status    |

+-----------+------------+---------+------------+---
------------+--------------+------------+-----------
--+-----------+
```

```
|      101 |         1 | Mumbai  | Lucknow
| 10:00:00      | 12:00:00     | Domestic    |
180 | Scheduled |

|      102 |         2 | kolkata | nagpur      |
12:00:00      | 04:00:00     | Domestic    |
180 | Scheduled |

|      104 |         4 | mumbai  | chennai     |
06:00:00      | 09:00:00     | Domestic    |
180 | Scheduled |

+-----------+------------+---------+-------------+---
-------------+--------------+-------------+-----------
--+-----------+

3 rows in set (0.00 sec)


mysql> select * from passenger;
```

```
+--------------+----------------+--------+------+----
---------+-------------+

| passenger_id | passenger_name |
gender | age  | passport_no | nationality
|

+--------------+----------------+--------+------+----
---------+-------------+

|        501 | Rahul Mehta    | M      |   35 |
P1234567    | Indian      |

|        678 | Rohit          | M      |   23 |
P85666      | Indian      |

|        888 | Jiya           | F      |   46 |
P45677      | Indian      |

+--------------+----------------+--------+------+----
---------+-------------+

3 rows in set (0.00 sec)
```

A **Trigger** is a set of SQL statements that automatically executes (fires) when a specific event occurs on a table.

Types of Triggers

1. **BEFORE INSERT**
2. **AFTER INSERT**
3. **BEFORE UPDATE**
4. **AFTER UPDATE**
5. **BEFORE DELETE**
6. **AFTER DELETE**

Triggers are used for:

- Data validation
- Logging changes
- Maintaining consistency

(Table for Logging) :-

```
CREATE TABLE booking_log (
   log_id INT AUTO_INCREMENT PRIMARY KEY,
   message VARCHAR(255)
);
```

1. BEFORE INSERT Trigger (Booking)

```
DELIMITER //
CREATE TRIGGER before_booking_insert
BEFORE INSERT ON booking
FOR EACH ROW
BEGIN
   IF NEW.seat_no IS NULL THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Seat number cannot be NULL';
   END IF;
END //
DELIMITER ;
```

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER before_booking_insert
    -> BEFORE INSERT ON booking
    -> FOR EACH ROW
    -> BEGIN
    ->     IF NEW.seat_no IS NULL THEN
    ->         SIGNAL SQLSTATE '45000'
    ->         SET MESSAGE_TEXT = 'Seat number cannot be NULL';
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> DELIMITER ;
mysql> INSERT INTO booking
    -> VALUES (4001, 101, 501, '2025-05-01', NULL, 'Economy');
ERROR 1644 (45000): Seat number cannot be NULL
```

2. AFTER INSERT Trigger (Booking Log)

```
DELIMITER //

CREATE TRIGGER after_booking_insert

AFTER INSERT ON booking

FOR EACH ROW

BEGIN

   INSERT INTO booking_log(message)

   VALUES (CONCAT('New booking added with ID: ', NEW.booking_id));

END //

DELIMITER ;
```

```
mysql> CREATE TRIGGER after_booking_insert
    -> AFTER INSERT ON booking
    -> FOR EACH ROW
    -> BEGIN
    ->     INSERT INTO booking_log(message)
    ->     VALUES (CONCAT('New booking added with ID: ', NEW.booking_id));
    -> END //
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> DELIMITER ;
mysql> INSERT INTO booking
    -> VALUES (5002, 101, 501, '2025-05-01', '21A', 'Economy');
Query OK, 1 row affected (0.04 sec)

mysql> SELECT * FROM booking_log;
+--------+--------------------------------+
| log_id | message                        |
+--------+--------------------------------+
|      1 | New booking added with ID: 5002 |
+--------+--------------------------------+
1 row in set (0.00 sec)
```

3. BEFORE UPDATE Trigger

```
DELIMITER //

CREATE TRIGGER before_booking_update

BEFORE UPDATE ON booking

FOR EACH ROW

BEGIN

   IF OLD.passenger_id <> NEW.passenger_id THEN
```

```
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Passenger ID cannot be changed';
    END IF;
END //
DELIMITER ;
```

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER before_booking_update
    -> BEFORE UPDATE ON booking
    -> FOR EACH ROW
    -> BEGIN
    ->     IF OLD.passenger_id <> NEW.passenger_id THEN
    ->         SIGNAL SQLSTATE '45000'
    ->         SET MESSAGE_TEXT = 'Passenger ID cannot be changed';
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> DELIMITER ;
mysql> UPDATE booking
    -> SET passenger_id = 999
    -> WHERE booking_id = 5002;
ERROR 1644 (45000): Passenger ID cannot be changed
```

4. AFTER UPDATE Trigger

```
DELIMITER //
CREATE TRIGGER after_booking_update
AFTER UPDATE ON booking
FOR EACH ROW
BEGIN
    INSERT INTO booking_log(message)
    VALUES (CONCAT('Booking updated: ', NEW.booking_id));
END //
DELIMITER ;
```

```
mysql> CREATE TRIGGER after_booking_update
    -> AFTER UPDATE ON booking
    -> FOR EACH ROW
    -> BEGIN
    ->     INSERT INTO booking_log(message)
    ->     VALUES (CONCAT('Booking updated: ', NEW.booking_id));
    -> END //
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> DELIMITER ;
mysql> UPDATE booking
    -> SET seat_no = '25B'
    -> WHERE booking_id = 5002;
Query OK, 1 row affected (0.02 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM booking_log;
+--------+--------------------------------+
| log_id | message                        |
+--------+--------------------------------+
|      1 | New booking added with ID: 5002 |
|      2 | Booking updated: 5002          |
+--------+--------------------------------+
2 rows in set (0.00 sec)
```

5. BEFORE DELETE Trigger

```
DELIMITER //

CREATE TRIGGER before_booking_delete

BEFORE DELETE ON booking

FOR EACH ROW

BEGIN

   IF OLD.travel_class = 'Business' THEN

      SIGNAL SQLSTATE '45000'

      SET MESSAGE_TEXT = 'Cannot delete Business class booking';

   END IF;

END //

DELIMITER ;
```

```
mysql> CREATE TRIGGER before_booking_delete
    -> BEFORE DELETE ON booking
    -> FOR EACH ROW
    -> BEGIN
    ->     IF OLD.travel_class = 'Business' THEN
    ->         SIGNAL SQLSTATE '45000'
    ->         SET MESSAGE_TEXT = 'Cannot delete Business class booking';
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> DELIMITER ;
mysql> INSERT INTO booking
    -> VALUES (5003, 101, 501, '2025-05-01', '10A', 'Business');
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> DELETE FROM booking
    -> WHERE booking_id = 5003;
ERROR 1644 (45000): Cannot delete Business class booking
```

6. AFTER DELETE Trigger

```
DELIMITER //

CREATE TRIGGER after_booking_delete

AFTER DELETE ON booking

FOR EACH ROW

BEGIN

   INSERT INTO booking_log(message)

   VALUES (CONCAT('Booking deleted: ', OLD.booking_id));

END //

DELIMITER ;
```

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER after_booking_delete
    -> AFTER DELETE ON booking
    -> FOR EACH ROW
    -> BEGIN
    ->     INSERT INTO booking_log(message)
    ->     VALUES (CONCAT('Booking deleted: ', OLD.booking_id));
    -> END //
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> DELIMITER ;
mysql> DELETE FROM booking
    -> WHERE booking_id = 5002;
Query OK, 1 row affected (0.03 sec)

mysql> SELECT * FROM booking_log;
+--------+--------------------------------+
| log_id | message                        |
+--------+--------------------------------+
|      1 | New booking added with ID: 5002 |
|      2 | Booking updated: 5002          |
|      3 | New booking added with ID: 5003 |
|      4 | Booking deleted: 5002          |
+--------+--------------------------------+
4 rows in set (0.00 sec)

mysql>
```

7. BEFORE INSERT Trigger (Passenger)

```
DELIMITER //

CREATE TRIGGER before_passenger_insert

BEFORE INSERT ON passenger

FOR EACH ROW

BEGIN

   IF NEW.age <= 0 THEN

      SIGNAL SQLSTATE '45000'

      SET MESSAGE_TEXT = 'Invalid age';

   END IF;

END //

DELIMITER ;
```

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER before_passenger_insert
    -> BEFORE INSERT ON passenger
    -> FOR EACH ROW
    -> BEGIN
    ->     IF NEW.age <= 0 THEN
    ->         SIGNAL SQLSTATE '45000'
    ->         SET MESSAGE_TEXT = 'Invalid age';
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> DELIMITER ;
mysql> INSERT INTO passenger
    -> VALUES (999, 'Test User', 'M', -5, 'P00000', 'Indian');
ERROR 1644 (45000): Invalid age
```

8. AFTER INSERT Trigger (Flight)

```
DELIMITER //

CREATE TRIGGER after_flight_insert

AFTER INSERT ON flight

FOR EACH ROW

BEGIN

   INSERT INTO booking_log(message)

   VALUES (CONCAT('New flight added: ', NEW.flight_id));

END //

DELIMITER ;
```

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER after_flight_insert
    -> AFTER INSERT ON flight
    -> FOR EACH ROW
    -> BEGIN
    ->     INSERT INTO booking_log(message)
    ->     VALUES (CONCAT('New flight added: ', NEW.flight_id));
    -> END //
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> DELIMITER ;
mysql> INSERT INTO flight
    -> VALUES (300, 1, 'Delhi', 'Goa', '10:00:00', '12:00:00', 'Domestic', 180, 'Scheduled');
Query OK, 1 row affected (0.03 sec)

mysql> SELECT * FROM booking_log;
+--------+--------------------------------+
| log_id | message                        |
+--------+--------------------------------+
|      1 | New booking added with ID: 5002 |
|      2 | Booking updated: 5002          |
|      3 | New booking added with ID: 5003 |
|      4 | Booking deleted: 5002          |
|      5 | New flight added: 300          |
+--------+--------------------------------+
5 rows in set (0.00 sec)
```

9. BEFORE UPDATE Trigger (Flight)

```
DELIMITER //

CREATE TRIGGER before_flight_update

BEFORE UPDATE ON flight

FOR EACH ROW

BEGIN

   IF NEW.total_seats < 0 THEN

      SIGNAL SQLSTATE '45000'

      SET MESSAGE_TEXT = 'Seats cannot be negative';

   END IF;

END //

DELIMITER ;
```

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER before_flight_update
    -> BEFORE UPDATE ON flight
    -> FOR EACH ROW
    -> BEGIN
    ->     IF NEW.total_seats < 0 THEN
    ->         SIGNAL SQLSTATE '45000'
    ->         SET MESSAGE_TEXT = 'Seats cannot be negative';
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> DELIMITER ;
mysql> UPDATE flight
    -> SET total_seats = -10
    -> WHERE flight_id = 101;
ERROR 1644 (45000): Seats cannot be negative
```

10. AFTER DELETE Trigger (Flight)

```
DELIMITER //

CREATE TRIGGER after_flight_delete

AFTER DELETE ON flight

FOR EACH ROW

BEGIN

   INSERT INTO booking_log(message)

   VALUES (CONCAT('Flight deleted: ', OLD.flight_id));

END //

DELIMITER ;
```

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER after_flight_delete
    -> AFTER DELETE ON flight
    -> FOR EACH ROW
    -> BEGIN
    ->     INSERT INTO booking_log(message)
    ->     VALUES (CONCAT('Flight deleted: ', OLD.flight_id));
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> DELIMITER ;
mysql> DELETE FROM flight
    -> WHERE flight_id = 300;
Query OK, 1 row affected (0.03 sec)

mysql> SELECT * FROM booking_log;
+--------+--------------------------------+
| log_id | message                        |
+--------+--------------------------------+
|      1 | New booking added with ID: 5002 |
|      2 | Booking updated: 5002          |
|      3 | New booking added with ID: 5003 |
|      4 | Booking deleted: 5002          |
|      5 | New flight added: 300          |
|      6 | Flight deleted: 300            |
+--------+--------------------------------+
6 rows in set (0.00 sec)
```

SHOW TRIGGERS;

SELECT * FROM booking_log;

```
mysql> SHOW TRIGGERS;

+------------------------+--------+-----------+------
-------------------------------------------------------
-------------------------------------------------------
----------------------------------------------+-------
-+-----------------------+---------------------------
-------------------------------------------------------
-----------------------------------+----------------+--
--------------------+---------------------+-----------
---------+

| Trigger              | Event  | Table     |
Statement
| Timing | Created              | sql_mode
| Definer       | character_set_client |
```

```
collation_connection | Database Collation
|

+------------------------+--------+-----------+------
-------------------------------------------------------
-------------------------------------------------------
----------------------------------------------+-------
-+-----------------------+---------------------------
-------------------------------------------------------
-----------------------------------+----------------+--
--------------------+---------------------+-----------
---------+

| before_booking_insert   | INSERT |
booking   | BEGIN
```

```
    IF NEW.seat_no IS NULL THEN

        SIGNAL SQLSTATE '45000'

        SET MESSAGE_TEXT = 'Seat number
cannot be NULL';

    END IF;

END                    | BEFORE | 2026-04-05
13:54:10.04 |
ONLY_FULL_GROUP_BY,STRICT_TRANS_TA
BLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,
ERROR_FOR_DIVISION_BY_ZERO,NO_ENGI
NE_SUBSTITUTION | root@localhost |
utf8mb4            | utf8mb4_0900_ai_ci   |
utf8mb4_0900_ai_ci |

| after_booking_insert    | INSERT |
booking   | BEGIN

    INSERT INTO booking_log(message)

    VALUES (CONCAT('New booking added
with ID: ', NEW.booking_id));

END                                    | AFTER  |
2026-04-05 14:07:24.94 |
ONLY_FULL_GROUP_BY,STRICT_TRANS_TA
BLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,
ERROR_FOR_DIVISION_BY_ZERO,NO_ENGI
NE_SUBSTITUTION | root@localhost |
utf8mb4            | utf8mb4_0900_ai_ci   |
utf8mb4_0900_ai_ci |

| before_booking_update   | UPDATE |
booking   | BEGIN

    IF OLD.passenger_id <>
NEW.passenger_id THEN

        SIGNAL SQLSTATE '45000'

        SET MESSAGE_TEXT = 'Passenger ID
cannot be changed';

    END IF;
```

```
END | BEFORE | 2026-04-05 14:09:27.50 |
ONLY_FULL_GROUP_BY,STRICT_TRANS_TA
BLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,
ERROR_FOR_DIVISION_BY_ZERO,NO_ENGI
NE_SUBSTITUTION | root@localhost |
utf8mb4            | utf8mb4_0900_ai_ci   |
utf8mb4_0900_ai_ci |

| after_booking_update    | UPDATE |
booking   | BEGIN

    INSERT INTO booking_log(message)

    VALUES (CONCAT('Booking updated: ',
NEW.booking_id));

END                                    |
AFTER  | 2026-04-05 14:10:44.14 |
ONLY_FULL_GROUP_BY,STRICT_TRANS_TA
BLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,
ERROR_FOR_DIVISION_BY_ZERO,NO_ENGI
NE_SUBSTITUTION | root@localhost |
utf8mb4            | utf8mb4_0900_ai_ci   |
utf8mb4_0900_ai_ci |

| before_booking_delete   | DELETE |
booking   | BEGIN

    IF OLD.travel_class = 'Business' THEN

        SIGNAL SQLSTATE '45000'

        SET MESSAGE_TEXT = 'Cannot delete
Business class booking';

    END IF;

END  | BEFORE | 2026-04-05 14:12:03.31
|
ONLY_FULL_GROUP_BY,STRICT_TRANS_TA
BLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,
ERROR_FOR_DIVISION_BY_ZERO,NO_ENGI
NE_SUBSTITUTION | root@localhost |
utf8mb4            | utf8mb4_0900_ai_ci   |
utf8mb4_0900_ai_ci |
```

```
| after_booking_delete    | DELETE |
booking   | BEGIN

   INSERT INTO booking_log(message)

   VALUES (CONCAT('Booking deleted: ',
OLD.booking_id));

END                                          |
AFTER  | 2026-04-05 14:13:15.68 |
ONLY_FULL_GROUP_BY,STRICT_TRANS_TA
BLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,
ERROR_FOR_DIVISION_BY_ZERO,NO_ENGI
NE_SUBSTITUTION | root@localhost |
utf8mb4            | utf8mb4_0900_ai_ci   |
utf8mb4_0900_ai_ci |

| after_flight_insert     | INSERT | flight    |
BEGIN

   INSERT INTO booking_log(message)

   VALUES (CONCAT('New flight added: ',
NEW.flight_id));

END                                          |
AFTER  | 2026-04-05 14:16:07.72 |
ONLY_FULL_GROUP_BY,STRICT_TRANS_TA
BLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,
ERROR_FOR_DIVISION_BY_ZERO,NO_ENGI
NE_SUBSTITUTION | root@localhost |
utf8mb4            | utf8mb4_0900_ai_ci   |
utf8mb4_0900_ai_ci |

| before_flight_update    | UPDATE | flight
| BEGIN

   IF NEW.total_seats < 0 THEN

      SIGNAL SQLSTATE '45000'

      SET MESSAGE_TEXT = 'Seats cannot
be negative';

   END IF;

END                    | BEFORE | 2026-04-05
14:17:27.38 |
ONLY_FULL_GROUP_BY,STRICT_TRANS_TA
BLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,
ERROR_FOR_DIVISION_BY_ZERO,NO_ENGI
NE_SUBSTITUTION | root@localhost |
utf8mb4            | utf8mb4_0900_ai_ci   |
utf8mb4_0900_ai_ci |

| after_flight_delete     | DELETE | flight
| BEGIN

   INSERT INTO booking_log(message)

   VALUES (CONCAT('Flight deleted: ',
OLD.flight_id));

END                                          |
AFTER  | 2026-04-05 14:18:36.52 |
ONLY_FULL_GROUP_BY,STRICT_TRANS_TA
BLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,
ERROR_FOR_DIVISION_BY_ZERO,NO_ENGI
NE_SUBSTITUTION | root@localhost |
utf8mb4            | utf8mb4_0900_ai_ci   |
utf8mb4_0900_ai_ci |

| before_passenger_insert | INSERT |
passenger | BEGIN

   IF NEW.age <= 0 THEN

      SIGNAL SQLSTATE '45000'

      SET MESSAGE_TEXT = 'Invalid age';

   END IF;

END                                  | BEFORE |
2026-04-05 14:15:02.63 |
ONLY_FULL_GROUP_BY,STRICT_TRANS_TA
BLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,
ERROR_FOR_DIVISION_BY_ZERO,NO_ENGI
NE_SUBSTITUTION | root@localhost |
utf8mb4            | utf8mb4_0900_ai_ci   |
utf8mb4_0900_ai_ci |

+-------------------------+---------+------------+------
-------------------------------------------------------
-------------------------------------------------------
```

```
----------------------------------------------+-------
-+-----------------------+---------------------------
-----------------------------------------------------
-----------------------------------+----------------+--
-------------------+---------------------+-----------
---------+

10 rows in set (0.04 sec)


mysql> SELECT * FROM booking_log;

+--------+--------------------------------+

| log_id | message                        |

+--------+--------------------------------+

|      1 | New booking added with ID: 5002
|

|      2 | Booking updated: 5002          |

|      3 | New booking added with ID: 5003
|

|      4 | Booking deleted: 5002          |

|      5 | New flight added: 300          |

|      6 | Flight deleted: 300            |

+--------+--------------------------------+

6 rows in set (0.00 sec)
```

Case Study : Centralized Club Management System

```
mysql> use club_management;

Database changed

mysql> SHOW TABLES;

+--------------------------+

| Tables_in_club_management |

+--------------------------+

| attendance               |

| budget_request           |

| club                     |

| club_coordinator         |

| club_lead                |

| club_members             |

| event                    |

| reports                  |

| super_admin              |

+--------------------------+

9 rows in set (0.12 sec)


mysql> select * from attendance;

+---------------+-----------+----------+---------+

| attendance_id | member_id | event_id | status  |

+---------------+-----------+----------+---------+

|             2 |         2 |        1 | Absent  |

|             3 |         3 |        2 | Present |

|             4 |         4 |        3 | Present |

|             5 |         5 |        4 | Absent  |

+---------------+-----------+----------+---------+

4 rows in set (0.02 sec)


mysql> select * from budget_request;

+-----------+----------+----------------+---------+--
-------+------------+-----------------+

| budget_id | event_id | coordinator_id |
club_id | lead_id | submission |
approved_status |

+-----------+----------+----------------+---------+--
-------+------------+-----------------+

|         1 |        1 |            101 |       1 |     201
| 2026-04-01 | Pending        |

|         2 |        2 |            102 |       2 |     202
| 2026-04-05 | Approved       |

|         3 |        3 |            103 |       3 |     203
| 2026-04-07 | Pending        |

|         4 |        4 |            104 |       4 |     204
| 2026-04-10 | Approved       |

|         5 |        5 |            105 |       5 |     205
| 2026-04-12 | Pending        |

+-----------+----------+----------------+---------+--
-------+------------+-----------------+

5 rows in set (0.01 sec)


mysql> select * from club;
```

```
+---------+---------------+---------+----------------
+
| club_id | club_name     | lead_id |
coordinator_id |
+---------+---------------+---------+----------------
+
|       1 | Tech Club     |     201 |           101 |
|       2 | Cultural Club |     202 |           102
|
|       3 | Sports Club   |     203 |           103
|
|       4 | Music Club    |     204 |           104
|
|       5 | Art Club      |     205 |           105 |
+---------+---------------+---------+----------------
+
5 rows in set (0.01 sec)

mysql> select * from club_coordinator;
+----------------+------------------+-----------------
+----------+---------+
| coordinator_id | coordinator_name |
email          | password | club_id |
+----------------+------------------+-----------------
+----------+---------+
|            101 | Anita            |
anita@gmail.com | pass     |       1 |
|            102 | Priya            |
priya@gmail.com | pass     |       2 |
|            103 | Rohit            |
rohit@gmail.com | pass     |       3 |
|            104 | Sneha            |
sneha@gmail.com | pass     |       4 |
|            105 | Vikas            |
vikas@gmail.com | pass     |       5 |
+----------------+------------------+-----------------
+----------+---------+
5 rows in set (0.01 sec)

mysql> select * from club_lead;
+---------+-----------+------------------+----------
+---------+----------------+
| lead_id | lead_name | email            |
password | club_id | coordinator_id |
+---------+-----------+------------------+----------
+---------+----------------+
|     201 | Rahul     | rahul@gmail.com  |
pass     |       1 |           101 |
|     202 | Karan     | karan@gmail.com  |
pass     |       2 |           102 |
|     203 | Amit      | amit@gmail.com   |
pass     |       3 |           103 |
|     204 | Pooja     | pooja@gmail.com  |
pass     |       4 |           104 |
|     205 | Neeraj    | neeraj@gmail.com |
pass     |       5 |           105 |
+---------+-----------+------------------+----------
+---------+----------------+
5 rows in set (0.01 sec)

mysql> select * from club_members;
+------+--------+------------------+------------+----
---------+---------+---------+
```

```
| m_id | m_name | m_email          |
m_password | event_roll  | domain  |
club_id |

+------+--------+------------------+------------+----
---------+---------+---------+

|    1 | Aman   | aman@gmail.com   | 123
| Participant | AI      |       1 |

|    2 | Riya   | riya@gmail.com   | 123
| Volunteer   | Design  |       1 |

|    3 | Neha   | neha@gmail.com   | 123
| Participant | Dance   |       2 |

|    4 | Arjun  | arjun@gmail.com  | 123
| Player      | Cricket |       3 |

|    5 | Meera  | meera@gmail.com  | 123
| Singer      | Vocal   |       4 |

|    7 | Simran | simran@gmail.com | 123
| Participant | AI      |       1 |

+------+--------+------------------+------------+----
---------+---------+---------+

6 rows in set (0.00 sec)


mysql> select * from event;

+----------+----------------+------------+------------
+-----------+---------+---------+-----------+

| event_id | event_name     | start_date |
end_date   | report_id | lead_id | club_id
| budget_id |

+----------+----------------+------------+------------
+-----------+---------+---------+-----------+

|        1 | Hackathon      | 2026-04-01 |
2026-04-02 |      NULL |     201 |       1 |
NULL |

|        2 | Dance Fest     | 2026-04-05 |
2026-04-06 |      NULL |     202 |       2 |
NULL |

|        3 | Football Match | 2026-04-07 |
2026-04-08 |      NULL |     203 |       3 |
NULL |

|        4 | Music Night    | 2026-04-10 |
2026-04-11 |      NULL |     204 |       4 |
NULL |

|        5 | Art Exhibition | 2026-04-12 |
2026-04-13 |      NULL |     205 |       5 |
NULL |

+----------+----------------+------------+------------
+-----------+---------+---------+-----------+

5 rows in set (0.01 sec)


mysql> select * from reports;

+-----------+----------+---------+----------------+

| report_id | event_id | lead_id |
coordinator_id |

+-----------+----------+---------+----------------+

|         1 |        1 |     201 |            101 |

|         2 |        2 |     202 |            102 |

|         3 |        3 |     203 |            103 |

|         4 |        4 |     204 |            104 |

|         5 |        5 |     205 |            105 |

+-----------+----------+---------+----------------+

5 rows in set (0.01 sec)


mysql> select * from super_admin;
```

```
+----------+------------+-----------------+----------
+----------------+

| admin_id | admin_name | email           |
password | overall_budget |

+----------+------------+-----------------+----------
+----------------+

|       1 | Admin1     | admin1@gmail.com
| pass123  |      100000.00 |

|       2 | Admin2     | admin2@gmail.com
| pass123  |      150000.00 |
```

```
|       3 | Admin3     | admin3@gmail.com
| pass123  |      200000.00 |

|       4 | Admin4     | admin4@gmail.com
| pass123  |      120000.00 |

|       5 | Admin5     | admin5@gmail.com
| pass123  |      180000.00 |

+----------+------------+-----------------+----------
+----------------+

5 rows in set (0.01 sec)
```

1. BEFORE INSERT

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER before_member_insert
    -> BEFORE INSERT ON Club_members
    -> FOR EACH ROW
    -> BEGIN
    ->     INSERT INTO member_log(action_type, message)
    ->     VALUES ('BEFORE INSERT', CONCAT('Inserting: ', NEW.m_name));
    -> END //
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> DELIMITER ;
mysql> INSERT INTO Club_members VALUES
    -> (300, 'UserA', 'a@gmail.com', '123', 'Participant', 'AI', 1);
Query OK, 1 row affected (0.04 sec)
```

2.AFTER INSERT

```
mysql> DELIMITER ;
mysql> INSERT INTO Club_members VALUES
    -> (300, 'UserA', 'a@gmail.com', '123', 'Participant', 'AI', 1);
Query OK, 1 row affected (0.04 sec)

mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER after_member_insert
    -> AFTER INSERT ON Club_members
    -> FOR EACH ROW
    -> BEGIN
    ->     INSERT INTO member_log(action_type, message)
    ->     VALUES ('AFTER INSERT', CONCAT('Inserted: ', NEW.m_name));
    -> END //
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql> INSERT INTO Club_members VALUES
    -> (301, 'UserB', 'b@gmail.com', '123', 'Participant', 'AI', 1);
Query OK, 1 row affected (0.02 sec)
```

3. BEFORE UPDATE

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER before_member_update
    -> BEFORE UPDATE ON Club_members
    -> FOR EACH ROW
    -> BEGIN
    ->     INSERT INTO member_log(action_type, message)
    ->     VALUES ('BEFORE UPDATE', CONCAT('Updating: ', OLD.m_name));
    -> END //
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql> UPDATE Club_members
    -> SET domain = 'ML'
    -> WHERE m_id = 300;
Query OK, 1 row affected (0.02 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

4. AFTER UPDATE

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER after_member_update
    -> AFTER UPDATE ON Club_members
    -> FOR EACH ROW
    -> BEGIN
    ->     INSERT INTO member_log(action_type, message)
    ->     VALUES ('AFTER UPDATE', CONCAT('Updated: ', NEW.m_name));
    -> END //
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql> UPDATE Club_members
    -> SET domain = 'Web'
    -> WHERE m_id = 301;
Query OK, 1 row affected (0.02 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

5. BEFORE DELETE

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER before_member_delete
    -> BEFORE DELETE ON Club_members
    -> FOR EACH ROW
    -> BEGIN
    ->     INSERT INTO member_log(action_type, message)
    ->     VALUES ('BEFORE DELETE', CONCAT('Deleting: ', OLD.m_name));
    -> END //
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql> DELETE FROM Club_members WHERE m_id = 300;
Query OK, 1 row affected (0.02 sec)
```

6. AFTER DELETE

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER after_member_delete
    -> AFTER DELETE ON Club_members
    -> FOR EACH ROW
    -> BEGIN
    ->     INSERT INTO member_log(action_type, message)
    ->     VALUES ('AFTER DELETE', CONCAT('Deleted: ', OLD.m_name));
    -> END //
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql> DELETE FROM Club_members WHERE m_id = 301;
Query OK, 1 row affected (0.02 sec)
```

```
mysql> SELECT * FROM member_log;
+--------+---------------+------------------+---------------------+
| log_id | action_type   | message          | action_time         |
+--------+---------------+------------------+---------------------+
|      1 | BEFORE INSERT | Inserting: UserA | 2026-04-07 19:15:37 |
|      2 | BEFORE INSERT | Inserting: UserB | 2026-04-07 19:17:12 |
|      3 | AFTER INSERT  | Inserted: UserB  | 2026-04-07 19:17:12 |
|      4 | BEFORE UPDATE | Updating: UserA  | 2026-04-07 19:18:18 |
|      5 | BEFORE UPDATE | Updating: UserB  | 2026-04-07 19:19:26 |
|      6 | AFTER UPDATE  | Updated: UserB   | 2026-04-07 19:19:26 |
|      7 | BEFORE DELETE | Deleting: UserA  | 2026-04-07 19:20:32 |
|      8 | BEFORE DELETE | Deleting: UserB  | 2026-04-07 19:21:21 |
|      9 | AFTER DELETE  | Deleted: UserB   | 2026-04-07 19:21:21 |
+--------+---------------+------------------+---------------------+
9 rows in set (0.00 sec)
```

Conclusion

In this assignment, we successfully implemented various types of database triggers including BEFORE INSERT, AFTER INSERT, BEFORE UPDATE, AFTER UPDATE, BEFORE DELETE, and AFTER DELETE using the Airline Management System and the Centralized Club Management System. Triggers helped us automate actions in response to specific events on database tables.

We used triggers for data validation, preventing incorrect data entry, and maintaining data integrity. Additionally, we implemented logging mechanisms using log tables to track changes such as insertions, updates, and deletions. This helped in auditing and monitoring database activities effectively.

Overall, this assignment improved our understanding of event-driven programming in databases and demonstrated how triggers can be used to enforce business rules, ensure consistency, and automate tasks in real-world applicatio